Design of a Surgical Training Environment Using Virtual Reality (VR)

Graduation Project — Presentation

University of Hail — College of Computer Science and Engineering

A graduation project that provides a safe surgical training environment using virtual reality. The system offers guided wound suturing simulation with performance assessment and runs in the browser without expensive hardware.

Team Members

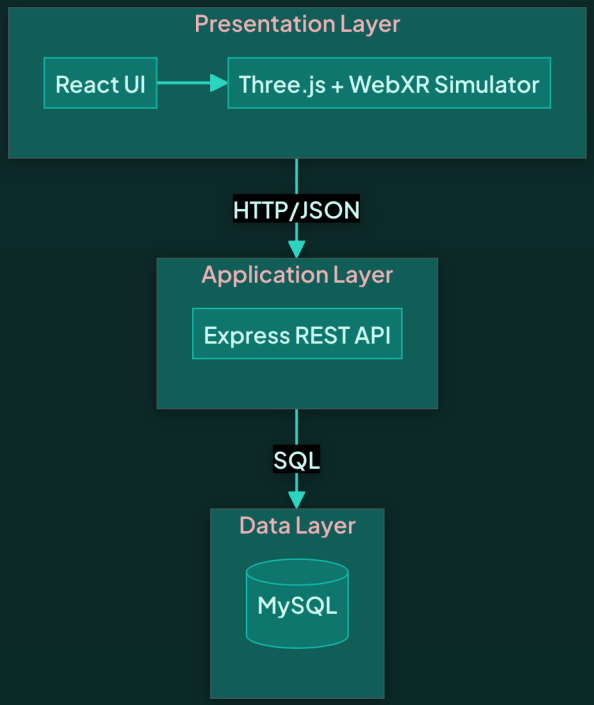
Fajr Khalid Mohammed Almuwallad	202103498
Hadeel Bader Alshammari	202104146
Mona Mohammad Thufel Alanazi	202004644
Wedad Nasser Alrshedi	202103422
Atheer Khalid Alshalan	202111183
Shouq Adel Alkhashman	202000258

A team of six members from the College of Computer Science and Engineering. Each member contributed to different aspects: system architecture, core functionality, backend and database, and frontend implementation.

Outline

The presentation covers all ten suggested topics: from system architecture and design, through functionality, backend, and frontend, then testing, integration, deployment, and finally achieved milestones and remaining work.

1. System Architecture and Design Implementation



Tier	Technology	Responsibility
Presentation	React 18, Vite 5, Tailwind, Three.js	UI, routing, 3D simulator, VR/AR
Application	Express (Node.js)	REST API, business logic, session management
Data	MySQL	TRAINEES, COURSES, SESSIONS, REPORTS

The system is built on a three-tier architecture: presentation layer (React and Three.js for UI and simulation), application layer (Express for request handling), and data layer (MySQL). Routing via React Router, state management via AuthContext and LangContext, with route protection for users and instructors.

2. Core Functionality Implementation

Feature	Implementation
Login & Register	AuthContext, api/auth/login, api/auth/register
User Profile	Specialty, PriorSimulationExperience, UnityUnrealExperience
Trainee Dashboard	Stats, chart, quick actions — api/stats/:traineeId
Instructor Dashboard	View all trainees and reports
14-Step Simulator	gameLogic.js, scene.js — proximity validation, hold-time by difficulty
VR & AR Mode	WebXR immersive-vr, immersive-ar — vr.js
Reports	Filter, CSV export, print
Bilingual	LangContext, i18n/strings.ts — Arabic RTL

Login and registration are implemented with user profile (Specialty, PriorSimulationExperience, UnityUnrealExperience). The trainee dashboard displays stats and charts. The simulator uses gameLogic.js and scene.js for proximity validation and hold-time by difficulty (Beginner 3.5s, Intermediate 2.5s, Advanced 1.5s). VR and AR modes via WebXR.

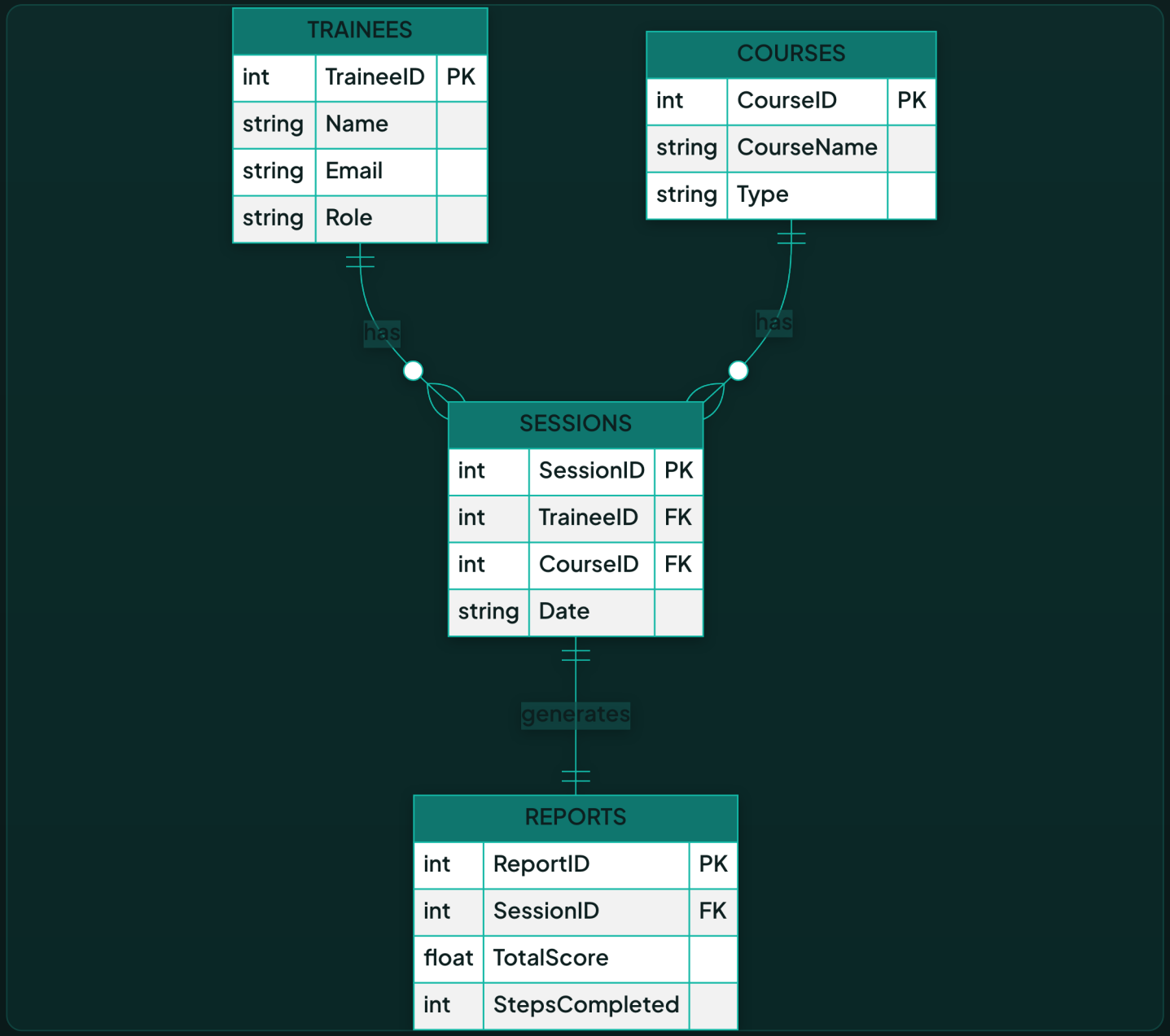
3. Backend and Database Implementation

EXPRESS API ENDPOINTS

Endpoint	Method	Purpose
/api/auth/register, /login	POST	Create account, authenticate
/api/sessions	POST	Create training session
/api/reports	POST, GET	Create and fetch reports
/api/admin/trainees, /reports	GET	Instructor: all trainees and reports

3NF. Indexes: TraineeID, Date, SessionID, Email. Migrations for profile, role, difficulty.

MYSQL SCHEMA — ER DIAGRAM



4. Frontend / User Interface Implementation

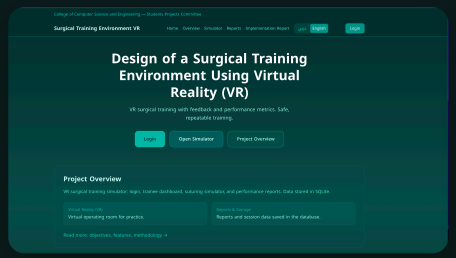
Technology	Purpose
React 18	Components, hooks, state
Vite 5	Build, dev server
Tailwind CSS 4	Utility-first styling
TypeScript	Type safety

PAGES & ROUTES

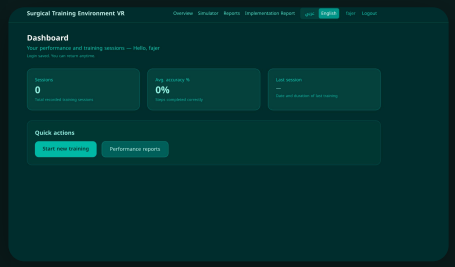
Landing (/), Overview (/overview), Login (/login), Dashboard (/dashboard), Training (/training), Reports (/reports), Instructor Dashboard (/instructor-dashboard), Implementation Report (/implementation-report)

UI Principles: Responsive, RTL for Arabic, teal color scheme, accessibility, error handling.

Landing Page



Dashboard



5. Testing Strategy and Methodology

UNIT TESTS

validation.ts: validateEmail, validatePassword, validateRequired — empty/invalid/valid cases

INTEGRATION TESTS

- Login Flow: /login → submit → redirect to /dashboard
- Register Flow: create account → redirect
- Training Flow: Login → /training → Start → simulator loads
- Reports Flow: complete session → /reports → CSV export, print

UI, SIMULATOR & DATABASE TESTS

Navigation, RTL, responsive, language switch. 14-step procedure, tools, localStorage. TRAINEES/SESSIONS/REPORTS CRUD.

Testing strategy is documented in docs/testing-spec.md. Unit tests cover validation functions (email, password, required fields). Integration tests cover login, register, training, and reports flows. UI tests include navigation, RTL layout, and responsiveness. Simulator and database tests cover step completion and CRUD operations.

6. Testing Results and Bug Fixing

Test Type	Description	Status
Unit	validation.ts — validateEmail, validatePassword	Pass
Integration	Login → Dashboard, Training → Simulator → Reports	Pass
UI	Navigation, RTL, responsive layout	Pass
Simulator	14-step procedure completion	Pass
Database	CRUD via API, MySQL persistence	Pass
VR/AR	WebXR session (when supported)	Pass

Bug Fixing: Issues resolved during development: validation edge cases, API error handling, simulator step sequencing, RTL layout adjustments.

All test categories passed. Bugs fixed during development: email validation edge cases, API 401 error handling, simulator hold-time logic, Arabic RTL layout adjustments. Results are documented in the graduation report.

7. System Integration Status

- **Frontend ↔ Backend:** React app communicates with Express API via fetch. Proxy configured in Vite (dev) for /api.
- **Backend ↔ Database:** Express uses mysql2 to connect to MySQL. All CRUD operations integrated.
- **Simulator ↔ App:** Standalone simulator.html receives sessionId, traineeId, difficulty via URL params. Reports saved to localStorage then synced to API.
- **Auth Flow:** Login/Register → AuthContext → localStorage → PrivateRoute/InstructorRoute.

Status: All components integrated and working together. End-to-end flow verified.

Integration is complete: React communicates with Express via fetch, and Express connects to MySQL. The simulator runs in a separate page, receives sessionId, traineeId, and difficulty via URL params, saves reports to localStorage, then syncs with the API. Auth flow works via AuthContext and localStorage with session persistence on page reload.

8. Deployment Readiness

Component	Command	Status
Frontend Build	`npm run build`	Vite produces dist/
Backend	`npm run server` or `cd server && npm start`	Express on port 3001
Database	`cd server && npm run init-db`	MySQL schema + migrations
Preview	`npm run preview`	Test production build locally

Environment: server/.env for MySQL (MYSQL_HOST, USER, PASSWORD, DATABASE). VITE_API_URL for production API URL.

Readiness: Build succeeds. Server runs. Database initializes. Ready for deployment to hosting (e.g., Vercel + Railway, or traditional VPS).

The npm run build command produces a dist folder ready for deployment. The server runs via npm run server. The database is initialized via init-db. The frontend can be deployed to static hosting (e.g., Vercel) and the backend to Node hosting (e.g., Railway), with managed MySQL. For production, bcrypt is recommended for password hashing.

9. Achieved Milestones and Current Progress

- ✅ Three-tier architecture implemented
- ✅ Login, Register, Auth with role-based access
- ✅ Trainee Dashboard with stats and chart
- ✅ Instructor Dashboard — all trainees and reports
- ✅ 14-step wound suturing simulator with proximity validation
- ✅ VR and AR modes (WebXR)
- ✅ Reports: filter, CSV export, print
- ✅ Bilingual (Arabic/English) with RTL
- ✅ MySQL backend with migrations
- ✅ Testing completed — all categories pass

Current Status: Project complete for graduation. All core features delivered.

All planned milestones were achieved: architecture, authentication and roles, trainee and instructor dashboards, 14-step simulator, VR and AR modes, reports with export and print, bilingual support, database with migrations, and testing. The project is ready for demonstration and delivery.

10. Remaining Work and Plan to Completion

REMAINING WORK (FUTURE ENHANCEMENTS)

- Additional surgical procedures (laparoscopic, knot-tying)
- Physical VR headset optimization
- AI-based performance assessment
- Session resume after interruption
- Password hashing (bcrypt) for production

PLAN TO COMPLETION

Graduation project scope: **Complete.** All required features delivered. Future work is optional enhancement for post-graduation or research.

The graduation project scope is complete. Remaining work is optional enhancement: additional surgical procedures, physical VR headset optimization, AI-based performance assessment, session resume after interruption. No blocking tasks for delivery.

References

- React Documentation — https://react.dev
- Three.js Documentation — https://threejs.org/docs
- WebXR Device API — https://www.w3.org/TR/webxr/
- Express.js — https://expressjs.com
- MySQL Documentation — https://dev.mysql.com/doc/
- Vite — https://vitejs.dev
- Tailwind CSS — https://tailwindcss.com

The project uses React, Three.js, WebXR, Express, MySQL, Vite, and Tailwind CSS. The links above are official documentation for each technology.

Thank You

Questions?

Surgical Training Environment VR — University of Hail